第6次作业总结

P339-340 题2

题目

For the digraph of Figure 6.15 obtain

(a) the in-degree and out-degree of each vertex

(b) its adjacency-matrix

(c) its adjacency-list representation

(d) its adjacency-multilist representation

(e) its strongly connected components

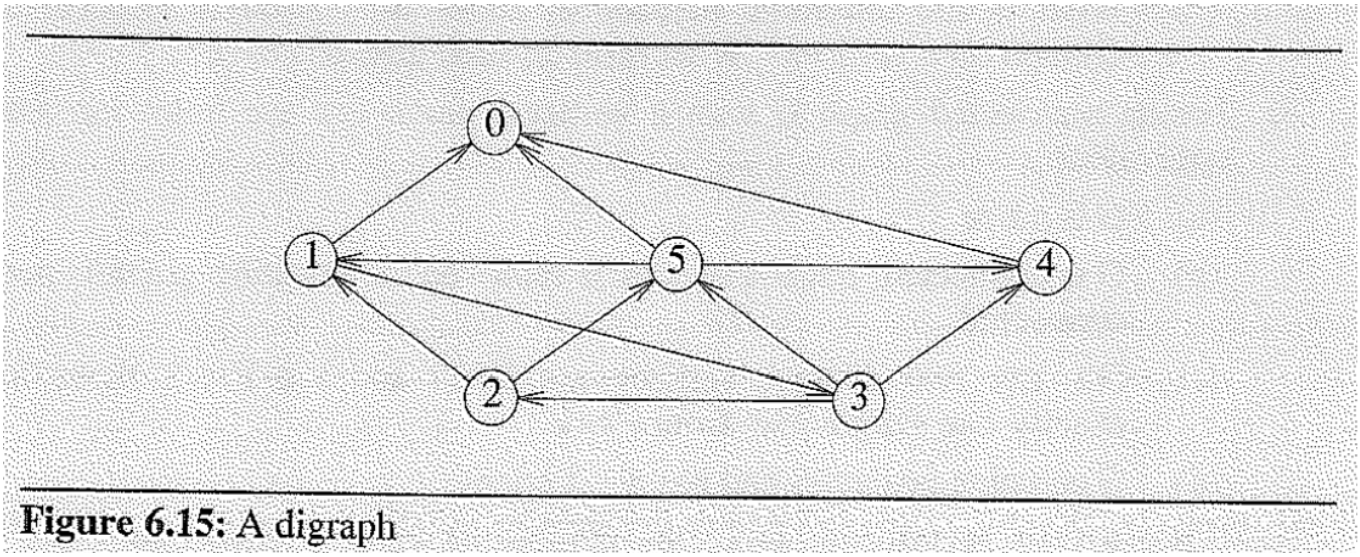


Figure 6.15: A digraph

参考答案

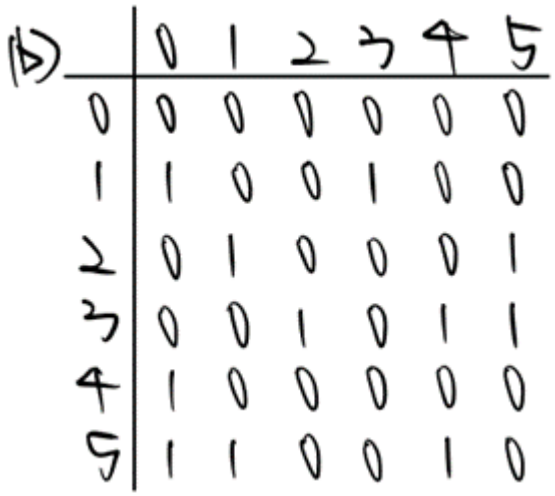
可参考如下解答:

(a) 每个顶点的入度和出度

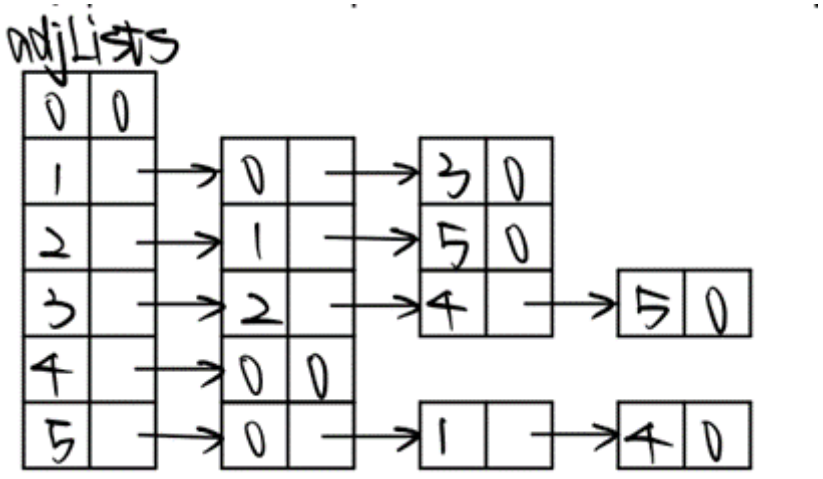
顶点	入度	出度
0	3	0

顶点	入度	出度
1	2	2
2	1	2
3	1	3
4	2	1
5	2	3

(b) 邻接矩阵

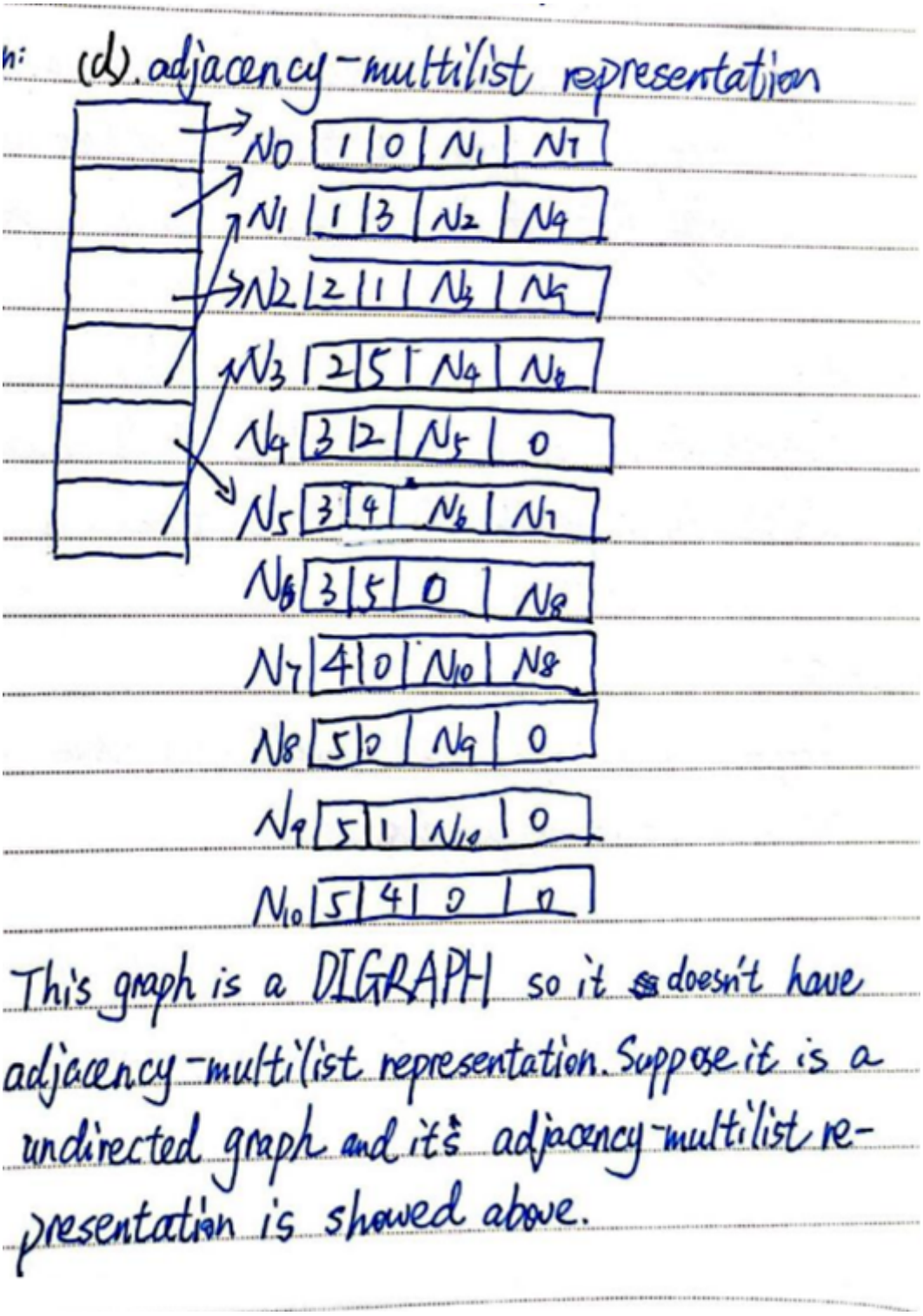


(c) 邻接表

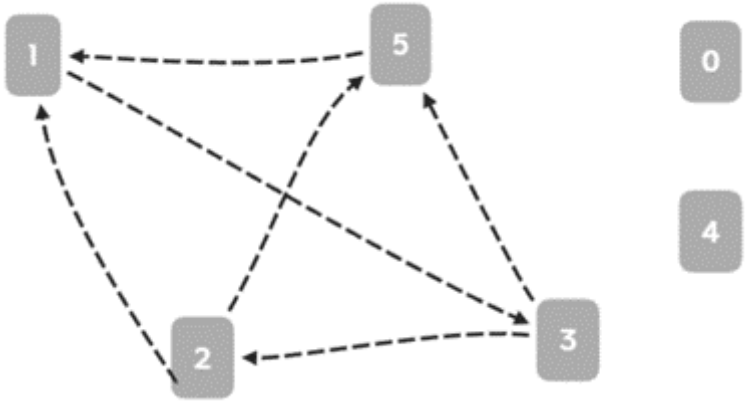


(d) 邻接多重表

这里题目给的是有向图，但是有向图的邻接多重表没有意义，暂且当作无向图来画


h: (d). adjacency-multilist representation
N0 1 0 N1 N7
N1 1 3 N2 N9
N2 2 1 N3 N5
N3 2 5 N4 N6
N4 3 2 N5 0
N5 3 4 N6 N7
N6 3 5 0 N8
N7 4 0 N10 N8
N8 5 2 N9 0
N9 5 1 N10 0
N10 5 4 0 0
This graph is a DIGRAPH so it doesn't have adjacency-multilist representation. Suppose it is a undirected graph and it's adjacency-multilist representation is showed above.

(e) 强连通分量



P339-340 题8

题目

For an undirected graph G with n vertices, prove that the following are equivalent:

(a) G is a tree

(b) G is connected, but if any edge is removed the resulting graph is notconnected

(c) For any two distinct vertices $u \in V(G)$ and $v \in V(G)$, there is exactly one simple path from u to v

(d) G contains no cycles and has $n-1$ edges

参考答案

可参考如下解答:

(a)->(b): 树上任意断开一条边会使得该边上的父亲节点与子节点不连通

(b)->(c): 反证：假设u到v存在两条路径，那么在其中一条路径上断边不会影响到联通性，矛盾。假设不成立。

(c)->(d): 数学归纳法证明：当n为1、2时，显然成立。

假设n为k时成立。

当n为$k+1$时，在k的基础上加一个点，为了保证任意两点的连通性，则需要连接新点与前k点任意一个点，即G有$(k-1)+1=k$个边，且无环，否则在前k点中任意连接两点，出现环，矛盾。

(d)->(a): n节点$n-1$条边无环图就是树

P372-373 题1

题目

Let T be a tree with root v. The edges of T are undirected. Each edge in T has a nonnegative length. Write a C++ function to determine the length of the shortest paths from v to the remaining vertices of T. Your function should have complexity $O(n)$, where n is the number of vertices in T. Show that this is the case.

参考答案

解题思路：题目给出的是一棵树，由于树中任意两点之间只有唯一一条路径，因此从根节点到任意节点的最短路径就是这条唯一路径。通过一次 DFS 或 BFS 累加边权即可计算所有最短路径，时间复杂度为 O(n)。

可参考如下解答：

```cpp
#include <iostream>
#include <vector>
using namespace std;

// 邻接表：每个元素是 (相邻节点, 边权)
vector<vector<pair<int, int>>> adj;

// 存储从根节点 v 到各点的最短距离
vector<long long> dist;

// 深度优先搜索
void dfs(int u, int parent) {
    for (auto &edge : adj[u]) {
        int v = edge.first;
        int w = edge.second;
        // 防止走回父节点
        if (v == parent) continue;
        // 更新距离
        dist[v] = dist[u] + w;
        dfs(v, u);
    }
}

int main() {
    int n;          // 节点数
    int root = 0;   // 根节点（假设是 0）
    cin >> n;

```

```
    adj.resize(n);
    dist.resize(n, 0);

    // 输入 n-1 条边
    for (int i = 0; i < n - 1; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }

    // 从根节点开始 DFS
    dfs(root, -1);

    // 输出结果
    for (int i = 0; i < n; i++) {
        cout << "从根节点 " << root << " 到节点 " << i
             << " 的最短路径长度为: " << dist[i] << endl;
    }

    return 0;
}
```

P372-373 题5

题目

Using the idea of $ShortestPath$ (Program 6.8), write a C++ function to find a minimum-cost spanning tree whose worst-case time is $O(n^2)$.

```
void MatrixWDigraph::ShortestPath(const int n, const int v){
    // dist[ljjl], 0≤j<n, is set to the length of the shortest path from v to j
    // in a digraph G with n vertices and edge lengths given by length[i][j]
    for(int i = 0; i < n; i++){ s[i] = false;  dist[i] = length[v][i];}// initializes
    s[v] = true;
    dist[v] = 0;

    for(i = 0; i < n-2; i++){// determine n-1 paths from vertex v
        int u = Choose(n);//choose returns a value u such that:
                          //distlul=mnimum distlwl where slwl-false
        s[u]= true;
        for(int w = 0; w < n; w++)
```

```
            if( !s[w] && dist[u] + length[u][w] < dist[w])
                dist[w] = dist[u] + length[u][w];
    }// end of for(i = 0;...)
// Program 6.8: Determining the lengths of the shortest paths
```

参考答案

解题思路：该算法本质上是 Prim 最小生成树算法的邻接矩阵实现，每一步从尚未加入生成树的顶点中选取与当前生成树连接代价最小的顶点，并利用该顶点更新其余顶点到生成树的最小连接代价。由于每一轮选择和更新操作都需要扫描全部顶点，因此算法在最坏情况下的时间复杂度为 $O(n^2)$。

可参考如下解答：

```
void MatrixWGraph::MinCostSpanningTree(int n) {
    const int INF = 1e9;
    bool s[100];         // s[i] = true 表示顶点 i 已在生成树中
    int lowcost[100];    // 当前生成树到 i 的最小代价
    int closest[100];    // 记录 i 是通过哪个顶点连接到生成树

    // 1. 初始化（从顶点 0 开始）
    for (int i = 0; i < n; i++) {
        s[i] = false;
        lowcost[i] = cost[0][i];
        closest[i] = 0;
    }
    s[0] = true;  // 将起点加入生成树

    // 2. 重复 n-1 次，选取最小代价边
    for (int i = 0; i < n - 1; i++) {
        int min = INF;
        int u = -1;

        // Choose: 找 lowcost 最小的未加入顶点
        for (int j = 0; j < n; j++) {
            if (!s[j] && lowcost[j] < min) {
                min = lowcost[j];
                u = j;
            }
        }

        // 将 u 加入生成树
        s[u] = true;
        cout << "选择边 (" << closest[u] << ", " << u
             << ") 权值为 " << lowcost[u] << endl;

```

```
33          // 3. 更新 lowcost
34          for (int w = 0; w < n; w++) {
35              if (!s[w] && cost[u][w] < lowcost[w]) {
36                  lowcost[w] = cost[u][w];
37                  closest[w] = u;
38              }
39          }
40      }
41  }
42
```

P389 题2

题目

(a) For the AOE network of Figure 6.44 obtain the early, $e()$, and late, $l()$, start times for each activity. Use the forward-backward approach

(b) What is the earliest time the project can finish?

(c) Which activities are critical

(d) Is there any single activity whose speed up would result in a reduction of the project length?

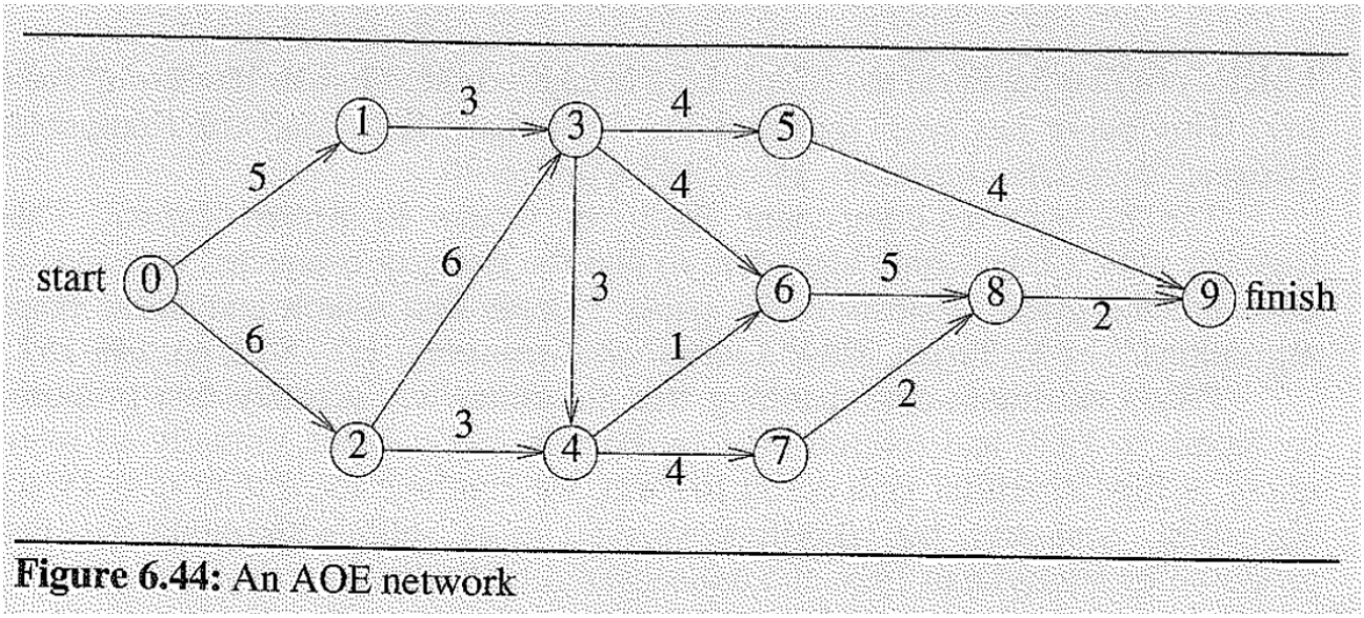


Figure 6.44: An AOE network

参考答案

可参考如下解答：

(a)

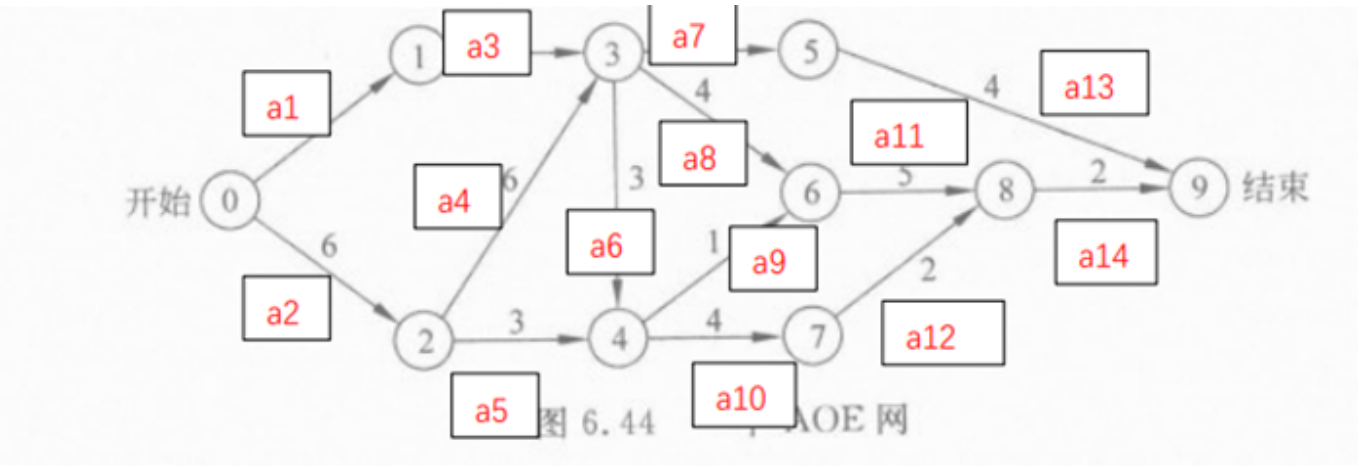


图 6.44 AOE 网

Event	E_e	E_l
E0	0	0
E1	5	9
E2	6	6
E3	12	12
E4	15	15
E5	16	19
E6	16	16
E7	19	19
E8	21	21
E9	23	23

Activity	e	l	l-e
a1	0	4	4

Activity	e	l	l-e
a2	0	0	0
a3	5	9	4
a4	6	6	0
a5	6	12	6
a6	12	12	0
a7	12	15	3
a8	12	12	0
a9	15	15	0
a10	15	15	0
a11	16	16	0
a12	19	19	0
a13	16	19	3
a14	21	21	0

(b) 最早可以在23天完成

(c) e和l相同的活动，<0,2> <2,3> <3,4> <3,6> <4,6> <4,7> <6,8> <7,8> <8,9>是关键的

(d) <0,2> <2,3> <8,9> 可以缩短工程周期。